FRED TALKS

22nd June 2026

CONTENTS

How big are our embeddings now and why?

★❤☆ VICKI BOYKIS ★❤☆ · 10 SEP 2025 · SOURCE

A few years ago, I wrote a paper on embeddings. At the time, I wrote that 200-300 dimension embeddings were fairly common in industry, and that adding more dimensions during training would create diminishing returns for the effectiveness of your downstream tasks (classification, recommendation, semantic search, topic modeling, etc.)

I wrote the paper to be resilient to changes in the industry since it focuses on fundamentals and historical context rather than libraries or bleeding edge architectures, but this assumption about embedding size is now out of date and worth revisiting in the context of growing embedding dimensionality and embedding access patterns.

As a quick review, embeddings are compressed numerical representations of a variety of features (text, images, audio) that we can use for machine learning tasks like search, recommendations, RAG, and classification. The size of the embedding is how many features our item has.

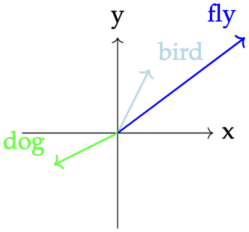
For example, let's say we have two butterflies. We can compare them among many dimensions, including wingspan, wing color, number of antennae. Let's say we have a 3-dimensional feature for a butterfly, it might look like this.

	wing_color	wing_span	antennae
butterfly_1	blue	10 cm	1
butterfly_2	red	20cm	2
butterfly_3	blue	15cm	1

Doing a visual vibe scan, we can eyeball the data and see that `butterfly_1` and `butterfly_3` are more similar to each other than to `butterfly_2` because their features are closer together.

But butterflies are 3-dimensional animals and some of these features are numerical. When we talk about embeddings in industry these days, we generally mean trying to understand the properties of text, so how does this concept work with words? We can't directly compare words like "bird" and "butterfly" or "fly" in a given text but we can compare their numerical representations if we map them into the same shared space. We can see that "bird" and "flying" are more similar to each other than to "dog" through their numerical representations.

Intuitively, we know this is true, but how do we artificially create these relationships?



There are different ways of creating text embeddings from a given word, all of which rely on analyzing that word in relation to the words around it in a given corpus of data. We can use traditional count-based approaches like TF-IDF based on term frequency in documents, or statistical approaches like PCA or LSA.

With the advent of deep learning models, we started learning representations generated from models like Word2Vec that maximized the probability that a left-out word would be next to other given words in the training dataset.

When we learn the embedding representation of a given word using probabilistic models, we are comparing how similar these words are to other words. Each feature is not an explicit feature like "wing color", but rather a vibe-based latent representation in the latent space that doesn't have a clear explanation.

For example, one dimension might be "this word is an action word" or maybe "this word is related to other words about food", but we generally don't know exactly what the model thinks each feature represents. In fact, this is a fascinating area of study we are just starting to understand how these latent representations work through ideas like control vectors, a concept that Anthropic explored in the famous Golden Gate Claude paper.

When we train a model, embedding size is initialized as a hyperparameter before model training, and we iterate on the size depending on our downstream evaluations after training. Picking the right hyperparameter is (alchemy) a combination of art and science and depends on optimizing training throughput, final embedding storage size, and the performance of the embedding on your downstream task both qualitatively and wrt to latency of performing search on embeddings of different sizes.

Since previous generations of models were smaller and trained in-house, hyperparameters were usually not published by companies, and as such, there was no standard agreement on embedding size. We generally, as an industry, understood that somewhere around 300 dimensions for a given embedding model might be enough to compress all the nuance of a given textual dataset. 300 was the number of dimensions typically used by earlier models like Word2Vec and GloVE.

After the publication of the attention paper BERT was released in 2018. This model's architecture introduced embeddings of 768 dimensions. Although previous RNN and LSTM models had been trained on GPUs, BERT was one of the first larger embedding models to be trained on GPUs (and TPUs), which meant that GPU optimization now became increasingly important.

The key behind training Transformer models efficiently is the ability to efficiently move data onto GPU and parallelize their matrix multiplication operations between several pipelines, aka attention heads, where each attention head can focus on understanding and defining a different part of the embedding feature space. As such, the embedding needs to be able to be partitioned evenly between the number of attention heads.

BERT has 12 attention heads, so 768 dimensions was selected from a combination of trial and error and efficiently parallelizing computation to "attend" to different parts of the feature space, meaning that, in model training, each head operates on a 64-dimensional subspace of the original 768-dimensional input embedding. This itself comes from the Transformer paper, where each sub-embedding size per head is commonly chosen as 64.

As a result, many BERT-style models, and related model families, used 768 as a baseline for embedding dimensions. Although it had a much larger training dataset, GPT-2 also implemented 768. And, although CLIP uses an embedding size of **768** as derived from the Vision Transformer architecture that CLIP uses for its image encoder, and for consistency, the text encoder also uses this dimension size[^1].

Even though training cycles for BERT were fairly small (4 days for the original BERT model) compared to the months-long pretraining processes that LLMs require these days, it was still hard computationally to infer these embedding sizes, even with GPU optimizations. For BERT, for example,

```
Finding the most similar pair in a collection of 10,000 sentences req
```

So in 2019, UKP created and optimized a model, SBERT, focusing on sentence-level representations, which unlocked faster inference, and Minilm became the standard baseline model for document-level chunk embeddings at 384 dimensions and still performs extremely well for a number of tasks.

What else changed was the rise of the open-source HuggingFace platform where model artifacts could be shared. Previously, they were either hosted in undiscoverable repos in GitHub or locked away in scientific repositories missing metadata. The rise of HuggingFace as a platform and the `transformers` library as a centralized API into training and inference for PyTorch-based models unlocked a level of standardization: now, many more people could simply download and replicate the existing models instead of rebuilding arcane research code from scratch. This led to much more standard embedding and architecture sizes used both in academia and industry.

Although 768 held for a fairly long time, with upwards pressure from market competition on model sizes as the result of the release of GPT-2 (the backbone of ChatGPT), we started seeing standard embedding sizes increase.

Part of this was the fact that companies no longer had to infer their own embeddings. Previously, embeddings had been custom-learned in labs or in companies whose core competency was processing large amounts of information that could be culled in retrieval through search or recommendation.

But now, with the advent of ChatGPT and API-based model availability, embeddings became a commodity available with a GET request, and the most popular embeddings became OpenAI's, which used 1536 dimensions, in line with GPT-3, which used much more training data than any previous model. (570GB versus GPT-2 which was 40GB, and 96 attention heads.)

It used to be the case that people only learned or fine-tune their own embeddings, but now all of the major AI providers have their own hosted sets of embeddings. Particularly common ones are OpenAI, with Cohere and Nomic close behind. Google also recently released embeddings for Gemini, with both API and local versions being available.

In addition to standardization via HuggingFace and APIs, MTEB, which benchmarks embeddings publicly, has grown as a resource where you can compare embedding models.

If we look at MTEB, the embedding benchmark today, we can see embedding sizes anywhere from our classic 768 to 4096 and beyond (note all of them are neatly divisible by 2, in line with architectural constraints)

Rank (Bor…	Model	Zero-shot	Embedding D…
1	gemini-embedding-001	99%	3072
2	Qwen3-Embedding-8B	99%	4096
3	Qwen3-Embedding-4B	99%	2560
4	Qwen3-Embedding-0.6B	99%	1024
5	Linq-Embed-Mistral	99%	4096
6	gte-Qwen2-7B-instruct	⚠ NA	3584
7	multilingual-e5-large-instruct	99%	1024
8	SFR-Embedding-Mistral	96%	4096
9	text-multilingual-embedding-002	99%	768
10	GritLM-7B	99%	4096
11	GritLM-8x7B	99%	4096

Finally, another consideration has been the change in the landscape with vector databases, which used to be huge, are now becoming more commoditized features in software and platforms like Postgres, S3, and Elasticsearch, leading to less out of the box necessity in vector storage and performance tuning.

With the constant upward pressure on embedding sizes not limited by having to train models in-house, it's not clear where we'll slow down: Qwen-3, along with many others is already at 4096. It does appear that we are beginning to trim down on growth. OpenAI implemented a concept called matryoshka representation learning that trains embeddings with the "most important" concepts first, and additional embeddings adding incremental gains, meaning that an embedding learned in 1024 dimensions might be just as useful in 64, as long as the first 64 dimensions compress most of the information efficiently, and also making sure we re-normalize them. There are also research that indicates that not all embeddings are necessary in retrieval + search tasks, and that we can in fact, in some cases, truncate 50% of them anyway.

It's been fascinating to watch the rise of dimensionality and the constant struggle in tradeoffs between creating ever-larger models and then the need for those embedding sizes to perform at inference and storage time, aka continuously coming up against the age-old machine learning tradeoff of recall versus precision, and the engineering tradeoff of hardware limitations versus software versus business considerations.

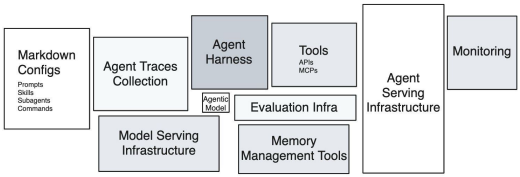
As these architectures have matured and internal models have become inference points of public-facing paid APIs, embeddings have gone from a mysterious byproduct of internal machine learning systems in companies with lots of data to a commodity used across many AI-powered applications across numerous application stacks.

All in all, it's been so much fun watching this space evolve, even if it means it looks like I'll be having to update my paper over and over again.

Hidden Technical Debt of AI Systems: Agent Runtime

HAN, NOT SOLO · 27 APR 2026 · SOURCE

Eleven years ago, Sculley et al. drew the diagram everyone in MLOps has seen: a tiny black box labeled "ML Code" surrounded by a sprawl of much larger boxes — data collection, feature extraction, configuration, monitoring, serving infrastructure. The point of the diagram was that the model code is the smallest piece of a real ML system, and that everything else is where the technical debt accumulates.



Hidden Technical Debt in Agentic AI Systems

The same diagram is being redrawn for agents. The agentic model call is the small box. The largest box to the right that is currently driving most of the spend and influencing how system architecture will be done in the futureincidents, is the **agent runtime**, or agent serving infrastructure.

The agent is not the model. The agent is the harness plus the model, running inside the runtime. And almost nobody is treating it that way.

What an Agent Runtime Actually Is

A agentic model by itself maps from text input to text output. Other modalities are in play, but the primary fabric is still text. An agent is what you get when you wrap the agentic model with harnesses so it can take actions, observe effects, feed those observation back into the next call. The execution environment where that happens is the runtime.

Concretely, an agent runtime is the union of:

- **compute substrate** — a container, microVM, or full VM where code runs.
- **filesystem** the agent can read and write, often with snapshot and rollback semantics.
- **tools** — shell, code interpreter, browser, file editor, MCP servers — exposed to the model as callable interfaces.
- **network boundary** that defines what the agent can reach and what can reach it.
- **state model** that decides what persists across turns, across episodes, and across users.
- **lifecycle controller** that starts, suspends, snapshots, resumes, and tears down environments.

In the taxonomy of RL environments for LLM agents, these are the (harness) and (state) components. In production, this is the part that decides whether your agent finishes a task in eight seconds or eight minutes, whether two users can share an environment safely, and whether a malicious prompt can read your secrets.

Most teams shipping agents today have not built this layer. They have rented it, glued it together from cloud primitives that were designed for a different workload, or simply not thought about it yet. That last group is the one accumulating the most debt.

Why Sandboxing Is Not Optional

Models hallucinate code. They run `rm -rf`. They paste credentials into curl commands. They follow instructions embedded in untrusted documents. They retry the same failing command in a tight loop and burn through a quota. None of these are exotic edge cases — they are the modal behavior of capable models when handed unrestricted tool access.

The sandbox is what stops these behaviors from becoming incidents. Four reasons it has to exist as a first-class layer rather than a hopeful disclaimer in the system prompt:

Isolation against the model's mistakes. A coding agent that mounts your repo and has shell access can, and eventually will, delete the wrong directory. Filesystem isolation, copy-on-write snapshots, and per-session ephemerality turn destructive actions into recoverable ones.

Isolation against prompt injection. The agent reads tool outputs. Tool outputs are attacker-controlled the moment the agent visits a webpage, opens a PDF, or processes a customer email. A sandbox is the only thing standing between an injected instruction and your production database. This is the agent-systems version of the SQL injection lesson, and we are at roughly the 2003 stage of learning it.

Multi-tenancy at training scale. RL training spins up thousands of concurrent rollouts. Each rollout needs its own filesystem, its own process tree, its own network namespace. Without strong isolation primitives, one rollout's flaky shell command takes down a neighbor's training step.

Reproducibility. A sandbox you can snapshot is a sandbox you can replay. Replay is how you debug a six-hour agent trajectory without re-running the whole thing. It is also how you turn a production failure into a regression test.

A web app's runtime can be sloppy because the user is the one driving and a refresh fixes most things. An agent's runtime cannot, because the agent will keep going long after a human would have stopped to ask a question.

The Cognition team is direct about this in their post-mortem on building Devin's cloud agent infrastructure: containerized agents share a kernel, and a single compromised session can reach every other container's filesystem, credentials, and network connections. Because agents generate their own code, run arbitrary commands, and probe the environment in unpredictable ways, kernel-escape is not a theoretical risk — it is the working assumption. Their conclusion, after more than a year of hypervisor engineering, is the same one the broader infrastructure community has converged on for any untrusted-code workload: VM-level isolation, where each session gets its own kernel and there is no shared attack surface.

Manus team has famously built and snapped shotted all their agent runtimes in a restorable format so users can come back in the future to replay what agents have done.

The Isolation Primitive Stack

Most of the differentiation between sandbox vendors lives one layer down, in the isolation primitive they pick. There are five primitives in serious use, with very different trade-offs.

Primitive	Isolation model	Cold start	Workload fit	Notable users
Linux containers (runc, Podman)	Shared host kernel, namespaces + cgroups + seccomp	~100ms	Trusted code, internal CI	Docker, Kubernetes default
Firecracker	KVM-based microVM, dedicated kernel per VM, ~5MB Rust VMM	~125ms boot, sub-second from snapshot	Untrusted code at high density	AWS Lambda, Fargate, E2B, Fly.io
gVisor	Userspace kernel intercepting syscalls; runc-compatible runtime	Container-class	Defense in depth where a microVM is overkill; GPU virtualization	Google Cloud Run, App Engine, Modal
Kata Containers	Lightweight VM per pod, OCI-compatible	Few hundred ms	Multi-tenant Kubernetes	Confidential Containers, some managed K8s
V8 isolates	Per-tenant JS heap inside a single process	Sub-millisecond	JavaScript-only, no arbitrary binaries	Cloudflare Workers, Deno Deploy

A few things are worth saying out loud about this table.

Containers are not a sandbox for agent code. They are a packaging and resource-control mechanism. The shared kernel means a kernel exploit, a misconfigured capability, or a sloppy seccomp profile takes down the isolation boundary. It is not okay for an agent that may execute attacker-controlled instructions hidden in a web page.

Firecracker is the de facto industry primitive for this category. AWS open-sourced it in 2018 to power Lambda and Fargate, and almost all agent-sandbox startup runs on top of it, including E2B, Fly.io, Vercel Sandbox. It boots a stripped-down Linux kernel inside KVM in roughly 125 milliseconds and uses about five megabytes of memory for the VMM itself. The combination of strong isolation, fast boot, high density is what makes thousands of concurrent rollouts economically viable.

gVisor is the middle path. It runs a userspace kernel that intercepts and re-implements Linux syscalls, giving you stronger isolation than namespaces without paying the full cost of a hardware VM. The trade-off is that some syscalls are slower or unsupported, which matters for workloads that hit the kernel hard. Google uses it for Cloud Run and App Engine; it is the right pick when you want defense-in-depth on top of containers but cannot move to microVMs.

Kata Containers fit a specific niche. They are an OCI-compatible runtime that wraps each pod in a lightweight VM, which lets you run untrusted workloads on Kubernetes without rewriting the orchestration layer. The cost is that you inherit Kubernetes' assumptions about pod lifetime, which do not match how agents actually behave.

V8 isolates are the wrong primitive for general agent workloads. They are extraordinary for JavaScript-only edge compute but an agent that needs to run arbitrary Python, install packages, drive a browser, or execute compiled binaries cannot live inside a V8 heap. Cloudflare's own Sandbox product addresses this by adding container-class compute alongside isolates.

The picks higher up the stack inherit these trade-offs. When a sandbox vendor advertises "VM-level isolation" they almost always mean Firecracker. When they advertise "millisecond cold starts" without VM isolation, they usually mean V8 isolates with a different threat model. The vendor brochure abstracts the primitive away; the threat model and the workload do not.

The Startup Sandbox Landscape

A category of "sandbox-as-a-service" companies has emerged in the last two years, almost all of them building on top of the primitives above. Northflank's roundup of Modal Sandboxes alternatives is a good market snapshot — the differentiation is in the developer ergonomics, the snapshot model, and the mix of tools that come pre-wired, not in the isolation layer underneath.

Vendor	Isolation	Snapshot model	Cold start	Notable design choice
Modal	gVisor	Filesystem diffs from base image	Sub-second from snapshot	GPU support
E2B	Firecracker microVM	Full VM snapshot	~150ms	Open-source SDK, language-agnostic, popular in the agent dev community
Daytona	Containers / VMs	Forkable workspaces	Few seconds	Forked dev environments, OCI-compatible
Northflank	Containers, GPU-aware	Persistent volumes	Container-class	GPU sandboxes for agents that train or run inference inside the loop
		Session replay	Seconds	

Browserbase / Steel / Hyperbrowser	Containerized browsers			Browser-only runtimes for web agents — DOM, screenshots, CDP
Cloudflare Workers Sandbox	V8 isolates + containers	Object snapshots	Milliseconds	Edge-first, shared-nothing model
Vercel Sandbox	Firecracker	Snapshot from build	Sub-second	Tied to Vercel's deploy and preview model

The reference Modal case study is Ramp Inspect: a background coding agent that now writes more than half of Ramp's merged pull requests, with each session running in its own Modal sandbox containing Postgres, Redis, Temporal, RabbitMQ, a VS Code server, and a VNC stack with Chromium. Filesystem snapshots are refreshed every thirty minutes by a cron, so a new session is working on a prompt within seconds. The lesson buried in that architecture is that the agent's productivity is bounded by the runtime's startup time, not by the model's tokens-per-second.

The browser-runtime category exists because driving Chromium is its own engineering problem — CDP, profiles, residential IPs, captcha handling, session persistence — and no general-purpose sandbox has it built in. Expect this to consolidate into the general-purpose vendors over the next eighteen months, the same way headless browsers absorbed into the major test frameworks.

It is also worth noting what nobody on this list does themselves. None of these vendors writes their own hypervisor. They all stand on top of the primitives in the previous section, which is what lets the category exist. The startups that have tried to build a complete stack — including the hypervisor — have ended up looking more like Cognition: years of engineering investment to get something that is finally indistinguishable from "we run on Firecracker with custom snapshot logic."

The Hyperscaler Sandbox Landscape

The hyperscalers got to this market late and are still catching up.

AWS Bedrock AgentCore ships a Code Interpreter and a Browser Tool as managed runtimes for agents. The isolation is microVM-class, the integrations point at the rest of the AWS data plane, and the pricing model is per-session. However, it has major gaps relative to the startups, where the runtime has to be coded and shipped like a Lambda function, instead of having the flexibility to support heterogeneous workloads, e.g, different agent harness + model combinations. Not a major problem for production, but a major problem for research and development.

Azure Container Apps Dynamic Sessions uses Hyper-V isolation and advertises sub-second start times for code interpreter sessions. It is the most directly comparable hyperscaler offering to E2B and Modal in terms of intent. The integration story is strongest if you are already on Azure OpenAI and AKS. It also suffers from the flexibility to support heterogeneous workloads.

GCP Cloud Run Sandboxes is ahead of the pack with the usability similar to Modal, E2B, Daytona and the likes. It's built on top of Cloud Run, which uses gVisor. Supports heterogeneous workloads.

Lambda, Fargate, App Runner, Cloud Run. It is tempting to use these as agent sandboxes because they are cheap and already in your account. They are designed for stateless, request-response workloads with strict execution-time limits and no native snapshot model. They will work for a demo. They will not work for an agent that runs for six hours, mutates a filesystem, and needs to fork mid-trajectory.

The hyperscaler offerings are converging on the right shape — microVM isolation, fast snapshots, managed tools — but the primitives underneath were built for web workloads. The startups had the advantage of building on top of those primitives without inheriting the legacy product surface area.

Experimentation and Production Want Different Runtimes

One of the major difference how AI engineering differes from traditional web development is the difference in development and production infrastructure requiremetns. Here's the difference between your training cluster (if you are training models), experimentation and evaluation runs, vs production workload.

Dimension	Experimentation / Training	Production / Serving
Concurrency shape	Thousands of parallel rollouts, bursty	One session per user, steady-state
Cold start tolerance	Critical — 5s × 10k rollouts is real money	Forgivable — users wait
State model	Fork, branch, replay, snapshot	Durable, per-user, auditable
Network policy	Often offline or recorded for determinism	Open internet, real APIs
Failure model	Drop the rollout, sample more	Retry, degrade, page someone
Observability	Full trace, every token, replayable	SLOs, error budgets, sampling

Cost model	Spot, preemptible, batch	Reserved, predictable, latency-bound
Determinism	Required for reproducible runs	Often counterproductive
Lifetime	Seconds to minutes	Minutes to hours, or pinned for a session

A training rollout wants to start in 200 milliseconds, run for thirty seconds against a frozen snapshot of the world, get scored, and disappear. A production session wants to start in two seconds, hold state for a forty-minute conversation, talk to the live internet, and survive a transient failure without losing the user's work.

The production side has a requirement that almost nobody surfaces in eval reports: the agent needs to survive the **async gaps** in real engineering work. Cognition's experience report is the cleanest articulation of this — an agent opens a PR, waits on CI, responds to a review comment, reruns tests, pushes a follow-up commit. Between each step there are minutes, hours, sometimes days where the agent's working state has to persist. A containerized agent can only survive these gaps by burning compute to stay alive, and if the container is rescheduled or times out the session is lost. Their solution was hypervisor-level snapshotting of the full machine state — memory, process tree, filesystem — so compute shuts down while the agent is idle and resumes exactly where it left off when a CI result arrives. Building that took longer than any other piece of infrastructure they had shipped to date.

Optimizing the same runtime for both training and production is how you end up with a system that is too slow for training and too brittle for production. Most teams discover this after the first time they try to "just deploy what we trained on" and find the latency budget eaten by a startup model that was fine when amortized over ten thousand rollouts and intolerable when paid by a single user staring at a spinner.

Dev/Prod Parity Is the Real Problem

In 2011, Twelve-Factor App told us to keep development, staging, and production as similar as possible. That advice was about reducing the gap between where the engineer types code and where the code runs. For agents, the gap that matters is between where the agent **trained** and **experimented** and where the agent **runs**.

An agent learns the runtime. Tool latencies, failure modes, shell quirks, filesystem layout, the exact way `ls` formats output, the way the browser resolves redirects. The model picks up all of it during training and bakes it into the policy, or the optimizer (RLM or GEPA) picks up all of it during training and bakes it into the harness or the prompts. Move these to a different runtime

and the behavior shifts. Tools that were idempotent become flaky. Commands that were instant now block. Snapshots that existed disappear. The agent has no abstract concept of “shell”; it has a concept of “the shell I trained on.”

This is a new flavor of distributional shift. It is not the data shift the MLOps community has been worrying about for a decade. It is **runtime shift**, and it shows up as silent quality regressions that no eval catches because the eval runs in the training runtime.

There are three honest paths through this:

Co-locate train and prod on the same runtime. Pick one sandbox provider, run RL rollouts on it, run production sessions on it, accept the lock-in. This is what the most disciplined AI engineering teams are doing.

Define a runtime contract and enforce it on both sides. A small, versioned interface — “this is the shell, these are the tools, these are the latencies, these are the failure modes” — that you implement once on your training infrastructure and once on your production infrastructure. The contract is the abstraction; the sandbox underneath can vary. This is harder than it sounds because the contract has to cover not just the API surface but the timing and failure semantics that the agent has learned.

Train against production noise. Inject latency, errors, and tool failures during training so the agent’s policy is robust to runtime variance instead of dependent on a specific runtime. Step-DeepResearch reports tangible gains from injecting 5–10% tool errors during training. This is the runtime analog of dropout.

The wrong answer, the one most teams are unconsciously picking, is to pick a sandbox for production as a software engineering solution, and forget all about the requirements from AI and machine learning, then spend the next quarters chasing flakiness of agent performances.

The Bill That Is Coming Due

Sculley’s argument was that ML systems accumulate technical debt in places that traditional software engineering does not have language for — feedback loops, configuration sprawl, undeclared consumers, glue code. Agent systems inherit all of that and add a new line item: the runtime debt.

Runtime debt looks like this. A team picks the sandbox that was easiest to integrate at prototype time. Production traffic exposes a different mix of failure modes. The team patches around the gap with retries, longer timeouts, and prompt-engineered apologies. The agent’s behavior gets

entangled with that runtime's quirks with whack-a-mole without disciplined research and development. A year later, switching runtimes is a six-month migration because nobody can predict which behaviors will move.

The agent has to live on a runtime. The teams that internalize that early will spend the next two years compounding the advantage. The teams that do not will spend the next two years paying down a debt they did not know they were taking on.

References

- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. NeurIPS. https://papers.nips.cc/paper_files/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract.html

- Lee, H. (2026). A Taxonomy of RL Environments for LLM Agents. Han, Not Solo. https://leehanchung.github.io/blogs/2026/03/21/rl-environments-for-llm-agents/

- Workman, G. (2026). How Ramp built a full-context background coding agent on Modal. Modal Blog. https://modal.com/blog/how-ramp-built-a-full-context-background-coding-agent-on-modal

- Lopopolo, R. (2026). Harness engineering: leveraging Codex in an agent-first world. OpenAI. https://openai.com/index/harness-engineering/

- Microsoft. (2024). Azure Container Apps Dynamic Sessions. https://learn.microsoft.com/en-us/azure/container-apps/sessions

- Firecracker. AWS open-source microVM. https://firecracker-microvm.github.io/

- Northflank. (2026). Top Modal Sandboxes Alternatives for Secure AI Code Execution. https://northflank.com/blog/top-modal-sandboxes-alternatives-for-secure-ai-code-execution


```
@article{
    leehanchung,
    author = {Lee, Hanchung},
    title = {Hidden Technical Debt of AI Systems: Agent Runtime},
    year = {2026},
    month = {04},
    day = {24},
```

```
    howpublished = {\url{https://leehanchung.github.io}},
    url = {https://leehanchung.github.io/blogs/2026/04/24/hidden-tech
}
```